⚡ CHEAT SHEET ÔN THI NHANH: PRJ301 FINAL EXAM

Tài liệu tóm tắt ngắn gọn các bảng so sánh và kiến thức cốt lõi giúp tra cứu nhanh trước khi thi.

---

1. HTTP Status Codes & Methods

Code	Ý nghĩa	Lỗi/Thành công	Ghi nhớ
200	OK	Thành công	Request thành công thường gặp
201	Created	Thành công	Tạo mới tài nguyên
302	Found	Chuyển hướng	Redirect tạm thời (`sendRedirect`)
400	Bad Request	Client Error	Sai cú pháp, parameter lỗi
401	Unauthorized	Client Error	Chưa đăng nhập (Authentication)
403	Forbidden	Client Error	Không có quyền truy cập (Authorization)
404	Not Found	Client Error	Sai URL, không thấy tài nguyên
405	Method Not Allowed	Client Error	Servlet không hỗ trợ GET/POST
500	Internal Error	Server Error	Lỗi runtime exception trong code

- **GET:** Truy xuất dữ liệu. Lộ dữ liệu trên URL. Idempotent & Safe.
- **POST:** Gửi dữ liệu. Ẩn dữ liệu trong Body. Không giới hạn size.

---

2. Servlet & JSP Lifecycle

- **Servlet Lifecycle:**

$$\text{Instantiation} \rightarrow \text{init(config)}\ [1\ \text{lần}] \rightarrow \text{service()}\ [\text{Mỗi request}] \rightarrow \text{destroy()}\ [1\ \text{lần}]$$

- **JSP Lifecycle:**\text{Translation (.jsp to .java)} \rightarrow \text{Compilation (.java to .class)} \rightarrow \text{jspInit()} \rightarrow \text{_jspService()} \rightarrow \text{jspDestroy()}

---

3. JSP Scripting Elements

Loại	Cú pháp	Vị trí biên dịch trong Servlet	Dấu chấm phẩy (;) cuối
Declaration	`<%! ... %>`	Nằm ngoài method `_jspService()`, cấp class.	✅ Bắt buộc phải có `;`
Scriptlet	`<% ... %>`	Nằm bên trong method `_jspService()`.	✅ Bắt buộc phải có `;`

Loại	Cú pháp	Vị trí biên dịch trong Servlet	Dấu chấm phẩy (;) cuối
Expression	`<%= ... %>`	Nằm trong `out.print(...)`.	❌ **Tuyệt đối KHÔNG có** `;`

4. So sánh RequestDispatcher.forward() vs response.sendRedirect()

Tiêu chí	`forward()`	`sendRedirect()`
Nơi xử lý	Phía Server (Nội bộ)	Phía Client (Browser redirect)
URL trên browser	**Không thay đổi**	**Thay đổi**
Số Request/Response	1 Request duy nhất	2 Request độc lập
Request Scope Data	**Giữ nguyên**	**Mất sạch**
Phạm vi	Chỉ trong cùng web app	Bất kỳ URL nào (kể cả ngoài app)
Cú pháp Java	`req.getRequestDispatcher("url").forward(req, res)`	`res.sendRedirect("url")`

5. So sánh Cookies vs HttpSession

Tiêu chí	Cookies	HttpSession
Nơi lưu trữ	Phía Client (Trình duyệt)	Phía Server (Bộ nhớ RAM)
Kích thước tối đa	Giới hạn 4KB mỗi cookie	Không giới hạn (tùy RAM server)
Độ bảo mật	Thấp (Dễ bị can thiệp)	Cao
Thời gian sống	Cấu hình qua `setMaxAge` (giây)	Cấu hình qua `session-timeout` (phút)
Mục đích dùng	Lưu cấu hình, theme, remember login	Lưu trạng thái đăng nhập, giỏ hàng

6. Cookies Max-Age Config (`setMaxAge(seconds)`)

- `seconds > 0`: Persistent Cookie (Lưu trên ổ cứng client trong `seconds` giây).
- `seconds < 0`: Session Cookie (Lưu trên RAM trình duyệt, bị xóa ngay khi đóng browser).
- `seconds == 0`: Ra lệnh cho trình duyệt **XÓA** cookie này ngay lập tức.

7. 9 JSP Implicit Objects vs 11 EL Implicit Objects

- **9 JSP Implicit Objects:** `request`, `response`, `session`, `application`, `out`, `config`, `pageContext`, `page`, `exception`.

- **11 EL Implicit Objects (Tất cả là Map ngoại trừ pageContext):** `pageScope`, `requestScope`, `sessionScope`, `applicationScope`, `param`, `paramValues`, `header`, `headerValues`, `cookie`, `initParam`, `pageContext`.

---

8. Các thẻ JSTL Core (`c:`) phổ biến

- `<c:out value="${var}" default="N/A" escapeXml="true" />`**:** Hiển thị an toàn, tự động escape HTML chống XSS.
- `<c:set var="name" value="val" scope="session" />`**:** Đặt giá trị cho biến ở một scope.
- `<c:remove var="name" scope="session" />`**:** Xóa biến khỏi scope.
- `<c:if test="${cond}">...</c:if>`**:** Thẻ kiểm tra điều kiện (⚠️ Không có thẻ else).
- `<c:choose> / <c:when test="${cond}"> / <c:otherwise>`**:** Rẽ nhánh nhiều điều kiện (If-Else).
- `<c:forEach var="item" items="${list}" varStatus="status">`**:** Duyệt mảng.
  - `status.index`: Bắt đầu từ 0.
  - `status.count`: Bắt đầu từ 1.
- `<c:url value="/path"><c:param name="k" value="v"/></c:url>`**:** Tạo URL an toàn tự động chứa Context Path.

---

9. 5 Bước lập trình JDBC

1. **Load Driver:** `Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver")`
2. **Get Connection:** `conn = DriverManager.getConnection(url, user, pass)`
3. **Prepare Statement:** `ps = conn.prepareStatement(sql)`
4. **Execute & Process:** `rs = ps.executeQuery()` (SELECT) hoặc `rows = ps.executeUpdate()` (INSERT/UPDATE/DELETE).
5. **Close Resources:** Đóng `rs` -> `ps` -> `conn` (Thứ tự ngược với lúc mở).

---

10. Filter & Listener Quick Rules

- **Filter:** Chặn và xử lý Request/Response.
  - Gọi `chain.doFilter(req, res)` để cho qua, không gọi = chặn đứng request.
  - Thứ tự chạy của các filter do thứ tự khai báo `<filter-mapping>` trong `web.xml` quyết định (annotation `@WebFilter` không đảm bảo thứ tự).
- **Listener:** Lắng nghe sự kiện (Tạo/Hủy context/session, thay đổi thuộc tính).
  - `ServletContextListener` thích hợp để khởi tạo/đóng DB connection pool.
  - `HttpSessionListener` dùng để đếm số user online.